

Java 單位換算器程式碼

https://claude.ai/code/session_019TTkqekVvYx36dkK3PTeK (Claude)

第一輪：沒有提供規格

User prompt: 仿刻一個「單位換算器」。

新增到 A1143314_OOP2_HW1

Response:

單位換算器已經寫好，存成 `hw3.java` 放進 `A1143314_OOP2_HW1`，也 push 上 GitHub 了。

你沒有給規格，所以我先照一般單位換算器的樣子做：

- 三種類別：
 - 長度：公里、公尺、公分、公釐、英里、碼、英尺、英吋
 - 重量：公噸、公斤、公克、台斤、磅、盎司
 - 溫度：攝氏、華氏、克耳文
- 操作方式：先選類別，輸入數值，選「從」和「換成」的單位，再按「換算」，或直接在輸入框按 Enter。「↔ 對調」可以把兩個單位互換。
- 防呆：輸入的不是數字時，會用紅字提示「請輸入正確的數字」，程式不會當掉。
- 換算方式：長度和重量都先換成基準單位（公尺、公斤），再換成目標單位。溫度不是單純乘一個倍數，所以先換成攝氏，再從攝氏換成目標單位。

我在虛擬螢幕上實際跑過幾組測試，結果都正確，例如：

- 12 英吋 = 30.48 公分
- 1 台斤 = 600 公克
- 100°C = 212°F
- -40°C = -40°F
- 0 K = -273.15°C

如果老師的簡報有指定規格，例如只要長度、介面要長什麼樣子，可以把那頁傳給我，我照規格再改。

這題一樣要交 AI 對話紀錄的 PDF 和 reflection。你現在傳的這則提問和我的回答就是真實紀錄，要我做成 `Java 單位換算器程式碼.pdf` 的話，跟我說一聲就好。

AI 這一輪產生的檔案 `hw3.java`（第一版，之後已被取代）：

```
import javax.swing.*;
import java.awt.*;
import java.util.HashMap;
import java.util.Map;

public class hw3 extends JFrame {

    // 每個單位換算成「基準單位」要乘上的倍數（長度以公尺為基準，重量以公斤為基準）
    private static final Map<String, Double> LENGTH = new HashMap<>();
    private static final Map<String, Double> WEIGHT = new HashMap<>();
    private static final String[] TEMPERATURE = {"攝氏 °C", "華氏 °F", "克耳文 K"};

    static {
        LENGTH.put("公里 km", 1000.0);
        LENGTH.put("公尺 m", 1.0);
        LENGTH.put("公分 cm", 0.01);
        LENGTH.put("公釐 mm", 0.001);
        LENGTH.put("英里 mi", 1609.344);
        LENGTH.put("碼 yd", 0.9144);
        LENGTH.put("英尺 ft", 0.3048);
        LENGTH.put("英吋 in", 0.0254);
    }
}
```

```

        WEIGHT.put("公噸 t", 1000.0);
        WEIGHT.put("公斤 kg", 1.0);
        WEIGHT.put("公克 g", 0.001);
        WEIGHT.put("台斤", 0.6);
        WEIGHT.put("磅 lb", 0.45359237);
        WEIGHT.put("盎司 oz", 0.028349523125);
    }

    private final JComboBox<String> categoryBox = new JComboBox<>(new String[]{"長度", "重量", "溫度"});
    private final JTextField inputField = new JTextField("1");
    private final JComboBox<String> fromBox = new JComboBox<>();
    private final JComboBox<String> toBox = new JComboBox<>();
    private final JLabel resultLabel = new JLabel("請輸入數值後按「換算」", SwingConstants.CENTER);

    public hw3() {
        setTitle("單位換算器");
        setSize(420, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        // 中間：類別、數值、來源單位、目標單位
        JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
        form.setBorder(BorderFactory.createEmptyBorder(15, 20, 0, 20));
        form.add(new JLabel("類別："));
        form.add(categoryBox);
        form.add(new JLabel("數值："));
        form.add(inputField);
        form.add(new JLabel("從："));
        form.add(fromBox);
        form.add(new JLabel("換成："));
        form.add(toBox);

        // 下方：按鈕與結果
        JButton convertButton = new JButton("換算");
        JButton swapButton = new JButton("≡ 對調");
        JPanel buttons = new JPanel();
        buttons.add(convertButton);
        buttons.add(swapButton);

        resultLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 18));

        JPanel bottom = new JPanel(new BorderLayout());
        bottom.add(buttons, BorderLayout.NORTH);
        bottom.add(resultLabel, BorderLayout.CENTER);
        bottom.setBorder(BorderFactory.createEmptyBorder(0, 10, 15, 10));

        add(form, BorderLayout.CENTER);
        add(bottom, BorderLayout.SOUTH);

        categoryBox.addActionListener(e -> loadUnits());
        convertButton.addActionListener(e -> convert());
        inputField.addActionListener(e -> convert()); // 在輸入框按 Enter 也能換算
        swapButton.addActionListener(e -> swapUnits());

        loadUnits();
        setVisible(true);
    }

    // 依照目前選的類別，重新填入兩個單位下拉選單
    private void loadUnits() {
        String[] units;
        switch ((String) categoryBox.getSelectedItem()) {
            case "長度": units = LENGTH.keySet().toArray(new String[0]); break;
            case "重量": units = WEIGHT.keySet().toArray(new String[0]); break;
            default: units = TEMPERATURE;
        }
        fromBox.setModel(new DefaultComboBoxModel<>(units));
        toBox.setModel(new DefaultComboBoxModel<>(units));
        if (units.length > 1) {
            toBox.setSelectedIndex(1);
        }
        resultLabel.setText("請輸入數值後按「換算」");
        resultLabel.setForeground(Color.BLACK);
    }

    private void swapUnits() {
        int from = fromBox.getSelectedIndex();
        fromBox.setSelectedIndex(toBox.getSelectedIndex());
        toBox.setSelectedIndex(from);
        convert();
    }

    private void convert() {
        double value;
        try {
            value = Double.parseDouble(inputField.getText().trim());
        }
    }

```

```

    } catch (NumberFormatException ex) {
        resultLabel.setForeground(Color.RED);
        resultLabel.setText("請輸入正確的數字");
        return;
    }

    String category = (String) categoryBox.getSelectedItem();
    String from = (String) fromBox.getSelectedItem();
    String to = (String) toBox.getSelectedItem();

    double result;
    if (category.equals("溫度")) {
        result = convertTemperature(value, from, to);
    } else {
        Map<String, Double> table = category.equals("長度") ? LENGTH : WEIGHT;
        result = value * table.get(from) / table.get(to);    // 先換成基準單位，再換成目標單位
    }

    resultLabel.setForeground(Color.BLACK);
    resultLabel.setText(format(value) + " " + from + " = " + format(result) + " " + to);
}

// 溫度不是單純乘倍數，所以先換成攝氏，再從攝氏換成目標單位
private double convertTemperature(double value, String from, String to) {
    double celsius;
    if (from.startsWith("華氏")) {
        celsius = (value - 32) * 5 / 9;
    } else if (from.startsWith("克耳文")) {
        celsius = value - 273.15;
    } else {
        celsius = value;
    }

    if (to.startsWith("華氏")) {
        return celsius * 9 / 5 + 32;
    } else if (to.startsWith("克耳文")) {
        return celsius + 273.15;
    } else {
        return celsius;
    }
}

// 最多顯示 6 位小數，並去掉多餘的 0 (例如 2.500000 → 2.5)
private String format(double d) {
    String s = String.format("%.6f", d);
    s = s.replaceAll("0+$", "").replaceAll("\\.$", "");
    return s.equals("-0") ? "0" : s;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(hw3::new);
}
}

```

第二輪：提供簡報與 Exercise 2 規格

User prompt: (附上簡報檔 GUI_3.pptx)

仿刻一個「單位換算器」。

視窗 480×280 ，BorderLayout

NORTH：JComboBox 選換算類型（長度／重量／溫度）

CENTER：兩列，每列一個 JTextField + 一個 JComboBox（來源單位、目標單位）

切換換算類型時，兩個單位 JComboBox 的選項要跟著換

長度：公尺、公分、英吋、英尺

重量：公斤、公克、磅、盎司

溫度：攝氏、華氏、克氏

SOUTH：「換算」按鈕，結果顯示在第二個 JTextField

新增到 A1143314_OOP2_HW1，取代掉原有的上傳檔案，你需要依照課程中給予的簡報告訴我為什麼這裡是這樣寫、這裡不是這樣寫，有沒有那些可以延伸討論的觀念

請以這個為基礎，幫我產生：

程式碼（可執行）

AI對話截圖/紀錄（沒用AI要寫未使用），同一項作業若有多份檔案，請置於同一資料夾

Reflection（AI給的哪裡有問題，改了什麼，為什麼改）

Response:

這次照 Exercise 2（簡報第 57 頁）的規格重寫，並取代前一版的 hw3.java。下面的說明會對照你上傳的簡報〈GUI and Swing〉，標出依據的頁碼。

完整程式碼

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class hw3 extends JFrame implements ActionListener {

    // 各類型的單位名稱
    private static final String[] LENGTH_UNITS = {"公尺", "公分", "英吋", "英尺"};
    private static final String[] WEIGHT_UNITS = {"公斤", "公克", "磅", "盎司"};
    private static final String[] TEMP_UNITS = {"攝氏", "華氏", "克氏"};

    // 每個單位換算成基準單位要乘的倍數（長度以公尺為基準，重量以公斤為基準），順序和上面的名稱對應
    private static final double[] LENGTH_RATE = {1.0, 0.01, 0.0254, 0.3048};
    private static final double[] WEIGHT_RATE = {1.0, 0.001, 0.45359237, 0.028349523125};

    private final JComboBox<String> typeBox = new JComboBox<>(new String[]{"長度", "重量", "溫度"});
    private final JTextField inputField = new JTextField(12);
    private final JTextField outputField = new JTextField(12);
    private final JComboBox<String> fromBox = new JComboBox<>(LENGTH_UNITS);
    private final JComboBox<String> toBox = new JComboBox<>(LENGTH_UNITS);
    private final JButton convertButton = new JButton("換算");

    public hw3() {
        setTitle("單位換算器");
        setSize(480, 280);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // NORTH：選換算類型
        JPanel north = new JPanel(); // JPanel 預設是 FlowLayout
        north.add(new JLabel("換算類型："));
        north.add(typeBox);

        // CENTER：兩列，每列一個 JTextField + 一個 JComboBox
        JPanel row1 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
```

```

row1.add(inputField);
row1.add(fromBox);

JPanel row2 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
outputField.setEditable(false); // 結果欄只顯示、不讓使用者輸入
row2.add(outputField);
row2.add(toBox);

JPanel center = new JPanel(new GridLayout(2, 1));
center.setBorder(BorderFactory.createEmptyBorder(20, 0, 0, 0));
center.add(row1);
center.add(row2);

// SOUTH: 換算按鈕
JPanel south = new JPanel();
south.add(convertButton);

add(north, BorderLayout.NORTH);
add(center, BorderLayout.CENTER);
add(south, BorderLayout.SOUTH);

toBox.setSelectedIndex(1); // 預設「公尺 → 公分」

// 向事件來源者註冊，由這個視窗物件 (this) 擔任傾聽者
typeBox.addActionListener(this);
convertButton.addActionListener(this);

setVisible(true); // 一定放最後
}

// 事件處理：用 getSource() 分辨是哪個元件觸發的
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == typeBox) {
        changeUnits();
    } else if (e.getSource() == convertButton) {
        convert();
    }
}

// 切換換算類型時，兩個單位 JComboBox 的選項跟著換
private void changeUnits() {
    String[] units;
    int type = typeBox.getSelectedIndex();
    if (type == 0) {
        units = LENGTH_UNITS;
    } else if (type == 1) {
        units = WEIGHT_UNITS;
    } else {
        units = TEMP_UNITS;
    }
    fromBox.setModel(new DefaultComboBoxModel<>(units));
    toBox.setModel(new DefaultComboBoxModel<>(units));
    toBox.setSelectedIndex(1);
    outputField.setText(""); // 類型換了，舊的結果就清掉
}

private void convert() {
    double value;
    try {
        value = Double.parseDouble(inputField.getText().trim());
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "請在第一個欄位輸入數字",
            "輸入錯誤", JOptionPane.ERROR_MESSAGE);
        return;
    }

    int type = typeBox.getSelectedIndex();
    int from = fromBox.getSelectedIndex();
    int to = toBox.getSelectedIndex();

    double result;
    if (type == 0) {
        result = value * LENGTH_RATE[from] / LENGTH_RATE[to];
    } else if (type == 1) {
        result = value * WEIGHT_RATE[from] / WEIGHT_RATE[to];
    } else {
        result = convertTemperature(value, from, to);
    }

    outputField.setText(format(result));
}

// 溫度不是乘一個倍數就好：先換成攝氏，再從攝氏換成目標單位 (0 攝氏、1 華氏、2 克氏)
private static double convertTemperature(double value, int from, int to) {
    double c;
    if (from == 1) {

```

```

        c = (value - 32) * 5 / 9;
    } else if (from == 2) {
        c = value - 273.15;
    } else {
        c = value;
    }

    if (to == 1) {
        return c * 9 / 5 + 32;
    } else if (to == 2) {
        return c + 273.15;
    } else {
        return c;
    }
}

// 最多顯示 4 位小數，並去掉多餘的 0 (例如 2.5000 → 2.5)
private static String format(double d) {
    String s = String.format("%.4f", d);
    s = s.replaceAll("0+$", "").replaceAll("\\.$", "");
    return s.equals("-0") ? "0" : s;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(hw3::new);
}
}

```

存成 `hw3.java`，用 `javac hw3.java` 編譯、`java hw3` 執行。我已經實際執行並測試過以下幾組換算，結果都正確：

輸入	結果
1 公尺 → 公分	100
12 英吋 → 公分	30.48
1 英尺 → 英吋	12
1 磅 → 公克	453.5924
16 盎司 → 磅	1
100 攝氏 → 華氏	212
32 華氏 → 克氏	273.15
-40 華氏 → 攝氏	-40

切換換算類型時，單位選單會跟著換成 4 個（長度、重量）或 3 個（溫度）選項。

逐段說明

1. 視窗設定

為什麼這樣寫

- `setSize(480, 280)` 對應規格的視窗尺寸。
- `setDefaultCloseOperation`、`setLocationRelativeTo(null)`、`setVisible(true)` 放最後，都是照簡報第 15 頁 `Demo01_HelloFrame` 的寫法。那頁也列出：拿掉 `setDefaultCloseOperation`，關掉視窗後程式還在跑；`setVisible(true)` 太早呼叫，元件會不見。

這裡不是這樣寫

- 第 15 頁的範例用的是 `setLayout(null)` 加 `setBounds()`，但 Exercise 2 規格指定 **BorderLayout**，所以這題不用絕對座標。
- 第 15 頁也提到，字型寫 "MS_JhengHei" 在 macOS 上會變豆腐字。所以這支程式沒有指定特定字型，直接用 Swing 預設字型，在 Windows 和 macOS 都能正常顯示中文。

2. BorderLayout：NORTH、CENTER、SOUTH

為什麼這樣寫

- 簡報第 18～20 頁：BorderLayout 要在 `add()` 裡指定位置，例如 `add(元件, BorderLayout.NORTH)`。第 20 頁也說明，這個 `add()` 是從父類別 `Container` 繼承來的 `add(Component comp, Object constraints)`。

- 規格的三個區域剛好對應 `BorderLayout.NORTH`、`CENTER`、`SOUTH`。

這裡不是這樣寫：為什麼不把元件直接加到 `NORTH`、`SOUTH`

- 第 18 頁提到，`BorderLayout` 裡的物件大小會根據視窗尺寸而定。從第 20 頁的範例截圖可以看到，放在 `North`、`South` 的按鈕被拉成跟視窗一樣寬。
- 如果直接 `add(convertButton, BorderLayout.SOUTH)`，「換算」按鈕就會變成一條長條。
- 所以先把元件放進 `JPanel` 再加入到 `BorderLayout`。`JPanel` 預設是 `FlowLayout`（第 21～23 頁的流動式版面配置），會讓元件維持原本大小並置中。這也呼應第 6 頁的「物件並非擺在 `JFrame` 層，而是置於 `JPanel`」。
- `NORTH` 另外加了一個「換算類型：」標籤，讓使用者知道下拉選單是做什麼用的。

3. `CENTER`：兩列，每列一個 `JTextField` + `JComboBox`

為什麼這樣寫

- 規格要求 `CENTER` 有「兩列」，所以用 `GridLayout(2, 1)` 把 `CENTER` 切成上下兩格，每格放一個 `JPanel`（`FlowLayout`），裡面再並排一個輸入框和一個單位選單。
- `new JTextField(12)` 用的是第 50 頁的建構元 `JTextField(int columns)`，設定文字方塊大約能容納 12 個字元寬。在 `FlowLayout` 裡，元件會照這個寬度顯示，不會被壓扁。

這裡不是這樣寫

- `GridLayout` 不在這份簡報裡（簡報介紹的是 `BorderLayout` 和 `FlowLayout`），這裡是延伸使用。如果想只用簡報教過的版面配置，也可以讓 `CENTER` 本身再用一個 `BorderLayout`，把第一列放在它的 `NORTH`、第二列放在 `SOUTH`。
- 不建議 `CENTER` 只用一個 `FlowLayout` 放四個元件。`FlowLayout` 會依視窗寬度自動換行（第 21 頁），能不能剛好排成「兩列」要看視窗大小，不穩定。
- 用 `new JTextField()`（不給 `columns`）也不好。在 `FlowLayout` 裡它的寬度會非常窄，幾乎打不了字。

4. 結果顯示在第二個 `JTextField`

為什麼這樣寫

- 規格要求結果顯示在第二個 `JTextField`，所以用 `outputField.setText(...)` 寫入結果。
- 加上 `outputField.setEditable(false)`，用的是第 49 頁 `JTextComponent` 的函數 `setEditable(boolean b)`。結果欄只拿來顯示，不讓使用者在裡面打字，避免看起來像是輸入框。

這裡不是這樣寫

- 不用 `JLabel` 顯示結果。`JLabel` 也能顯示文字，但規格指定第二個 `JTextField`。而且 `JTextField` 就算不可編輯，使用者還是可以選取、複製結果。

5. 事件處理：委派事件模式

為什麼這樣寫

- 照簡報第 24、28～31 頁的委派事件模式：
 - 類別寫 `implements ActionListener`，讓視窗物件自己擔任「事件傾聽者」（第 28 頁）。
 - 用 `typeBox.addActionListener(this)`、`convertButton.addActionListener(this)` 向事件來源者註冊（第 28 頁）。
 - 在 `actionPerformed(ActionEvent e)` 裡寫事件處理，而且要 `import java.awt.event.*`；（第 29 頁）。
- 這題有兩個事件來源者：換算類型選單和換算按鈕，共用同一個傾聽者。所以要用 `e.getSource()` 判斷是誰觸發的，再決定是切換單位還是換算。`JComboBox` 選擇改變時，也會像按鈕一樣送出 `ActionEvent`。

這裡不是這樣寫

- 前一版用的是 `lambda (e -> convert())`。功能一樣，但簡報教的是 `implements ActionListener` 加 `addActionListener(this)`，所以這版改成簡報的寫法，比較好對照課堂內容。
- 用 `e.getSource() == typeBox` 比較的是「是不是同一個物件」，所以這裡用 `==` 是正確的；跟字串比較不能用 `==` 是不同的情況。

延伸閱讀

- 第 33 頁的寫法：定義一個內部類別 implements ActionListener 當傾聽者。那頁的內部類別宣告成 static，是因為在 main 這種類別函數裡不能建立非 static 的內部類別物件。
- 第 25 頁標題的 Observer（觀察者）模式：委派事件模式其實就是 Observer 模式的應用，事件來源者是「被觀察者」，傾聽者是「觀察者」。

6. 切換類型時，單位選單跟著換

為什麼這樣寫

- 規格要求切換換算類型時，兩個單位選單的選項要跟著換。在 changeUnits() 裡，依照選到的類型拿出對應的單位陣列，用 setModel(new DefaultComboBoxModel<>(units)) 一次把整份選項換掉。
- 換完後把目標單位預設成第二個選項，並清空舊的結果，避免畫面上還留著上一種類型的換算結果。

這裡不是這樣寫

- 也可以用 removeAllItems() 再用迴圈 addItem() 一個一個加回去。這樣也能動，但每次刪除、新增都可能觸發選單的事件；如果之後有人幫單位選單加上傾聽者，就會被觸發很多次。用 setModel 一次換掉比較乾淨。
- 不要每次 new 一個新的 JComboBox。畫面上的還是舊元件，要把舊的移除、新的加回去，再 revalidate()，麻煩又容易出錯。

延伸討論

- JComboBox 也可以用 ItemListener 監聽，但切換一次選項會觸發兩次 itemStateChanged（舊選項取消、新選項選取）。這題用 ActionListener 比較單純。

7. 換算邏輯

為什麼這樣寫

- **長度和重量**：先換成「基準單位」再換成目標單位，公式是 數值 × 來源倍數 ÷ 目標倍數。例如 12 英吋 → 公分：12 × 0.0254 ÷ 0.01 = 30.48。這樣 4 個單位只要記 4 個倍數，不用寫 4 × 4 = 16 條公式。
- 單位名稱陣列和倍數陣列的**順序一一對應**，所以用 getSelectedIndex() 拿到位置，就能直接查表，不用比對字串。
- **溫度**不是乘一個倍數就好（0°C 不等於 0°F），所以先換成攝氏，再從攝氏換成目標單位。

這裡不是這樣寫：容易出錯的地方

- 華氏換攝氏寫成 5 / 9 * (value - 32)，結果會**永遠是 0**。因為 5 / 9 是整數除法，算出來是 0。這裡寫成 (value - 32) * 5 / 9，value 是 double，從左邊算過來就一直是小數運算，所以是對的。
- 如果之後新增單位，名稱陣列和倍數陣列要放在**同一個位置**，不然會查到錯的倍數。
- 規格寫的是「克氏」，指的就是克耳文溫度（K），換算是攝氏 + 273.15。

8. 輸入檢查與結果格式

為什麼這樣寫

- Double.parseDouble(...) 遇到不是數字的輸入（例如空白或 abc）會丟出 NumberFormatException。用 try / catch 接住，跳出 JOptionPane 錯誤訊息，程式就不會當掉。
- 先 trim() 去掉前後空白，避免多打一個空格就被當成錯誤。
- 結果最多顯示 4 位小數，並去掉多餘的 0，例如顯示 30.48，而不是 30.480000。

這裡不是這樣寫

- 沒有 try / catch 的話，使用者輸入錯誤時，例外會印在主控台，畫面上卻沒有任何反應，使用者不知道發生什麼事。

可以延伸討論的觀念

1. **MVC（簡報第 58~64 頁）**：第 60 頁把 MVC 對應成 Model=Data、View=Layout、Controller=Listener。這支程式已經有一點 MVC 的雛形：
 - **Model**：單位陣列、倍數陣列和 convertTemperature()，這些都跟畫面無關。

- **View**：建構子裡排版面的程式。
- **Controller**：actionPerformed()。

下一步可以把換算邏輯抽成獨立的 `UnitConverter` 類別，讓它完全不碰 `Swing`。這樣換算公式可以單獨測試，以後改成網頁版或手機版也能直接沿用。

2. **浮點數誤差**：`double` 是二進位小數，`0.1 + 0.2` 會得到 `0.30000000000000004`。這也是結果要用 `String.format` 控制小數位數的原因。需要精確計算的場合（例如金額）會改用 `BigDecimal`。

3. **用 `enum` 管理單位**：把單位名稱和倍數綁在一起，寫成 `enum LengthUnit { METER("公尺", 1.0), ... }`，就不會發生兩個陣列順序對不上的問題。

4. **AWT vs Swing（第 3 頁）**：這支程式的元件都是 `Swing` 的（`JComboBox`、`TextField`、`Button`），但 `BorderLayout`、`FlowLayout`、`GridLayout` 和 `ActionEvent` 都來自 `java.awt`，正好呼應第 3 頁說的，現在 `AWT` 幾乎只用它的 `Color`、`Font`、`LayoutManager`。